



STRUCTURES

Chapter 8

Introduction to structure

Structure is another user defined data type available in C that allows:

- To **combine data items of different kinds/data types** and represent a form of record. And each data type has a name as a selection from the structure

- Or it can be defined as collection of one or more variables grouped together under a single name for convenient handling (one name in a block of memory)

The variables in structure are called **members** and may have any type of data

Simply, where **arrays** hold data items of the same kind(similar data types), **structure** on other hand allows to combine data items of different kinds (different data types)

Defining structure

For example, As a student.

Your Name is a string

Your Reg number is integer

and your GSM is integers.

So, here Name, Reg No and GSM are different (different types of data).

Hence, **structure** comes in picture to **combine them under a single name** and represent a form of your record.

Structure declaration

The keyword **struct** (short form of structured data type) is used to define a new data of structure type followed by a **name** or **tag (identifier)** as follows

And the role of tag is to inform the compiler the template that a program **is interested in** and associated with its members

```
struct [structure tag/name] {  
    member definition(or data type);  
    member definition(or data type);  
    ...  
    member definition(or data type);  
};
```

The **structure tag** is a normal variable definition, such as `int ;` or `float ;` or any other valid variable definition

Syntax

```
struct structure_name {  
    // data_type variable 1  
    // data_type variable 2  
    // data_type variable 3  
    ...  
    ....  
};
```

A structure is defined with the *struct* keyword collecting primary data types and other structures.

- The *structure_name* holds the name we want to give to your structure.
- *data_type* variables is data types like *int*, *char*, *float*, *etc.*



example

- Let's create a data structure suitable for storing the date.

```
struct date {  
    int month;  
    int day;  
    int year; };
```

- Other data structures may be defined as consisting of the same composition as the date structure,
 - `struct date todays_date;`
 - defines a variable called `todays_date` to be of the same data type as that of the newly defined data type `struct date`.





Assigning values to structure elements

- To assign `today's_date` to the individual elements of the structure `today's_date`, the statement
 - `today's_date.day = 31;`
 - `today's_date.month = 8;`
 - `today's_date.year = 2006;`
- Note the use of the `.` element to reference the individual elements within `today's_date`.



Example #1

```
#include<stdio.h>
struct date      /* Global definition of type date*/
{
int day;
int month;
int year;
};
main()
{
struct date today;
today.day=31;
today.month=8;
today.year=2006;
printf("Today's date is: %d/%d/%d\n",today.day, today.month,
    today.year);
}
```



Another example

```
#include <stdio.h>
/* Created a structure here. The name of the structure is StudentData.
struct StudentData{
    char stu_name;
    char stu_college;
    int stu_Id;
};
int main(){
    struct StudentData student; /* student is the variable of structure StudentData*/
    student.stu_name = "Pastor Steven "; /*Assigning the values of each struct member
    student.stu_college = "UR_CST";
    student.stu_Id = 20056969;
    /* Displaying the values of struct members */
    printf("\nStudent Name is: %s", student.stu_name);
    printf("\nStudent college is: %s", student.stu_college);
    printf("\nStudent Id is: %d", student.stu_Id);
    return 0;}
```

Another example relies on string function

```
#include <stdio.h>
#include <string.h>
struct Book
{
    char title[100];
    char author[100];
    char subject[100];
    int book_id;
};
int main( ) {
    struct Book Book; /*structure get variable name in memory allocation*/
    /* another variable like Book specification as internal pointer variable for struct
Book to store and display information on screen*/
    strcpy( Book.title, "Computer Programming"); /* Pointing or access member */
    strcpy( Book.author, " Professor Engineer");
    strcpy( Book.subject, "C Language Tutorial");
    Book.book_id = 6495407;
    printf( "Book title : %s\n", Book.title);          /* print Book info */
    printf( "Book author : %s\n", Book.author);
    printf( "Book subject : %s\n", Book.subject);
    printf( "Book book_id : %d\n", Book.book_id);
```



Example #3

```
#include<stdio.h>
void main()
{
    struct employee
    {
        char fname[20],lname[20];
        int age,id;
        float weight,height;
    };

    struct employee emp;
    printf("Enter employee ID:\n");
    scanf("%d",&emp.id);
    printf("Enter employee First Name and Last Name\n");
    scanf("%s %s",&emp.fname,&emp.lname);
    printf("Enter employee age:\n");
    scanf("%d",&emp.age);
    printf("Enter the Employee weight and height:\n");
    scanf("%0.2f %0.2f",&emp.weight,&emp.height);

    print("Employee ID: %d", emp.id); ....
}
```



Defining different structure templates

- Structure templates are created by using the struct keyword

```
struct Date
{
    int  day;
    int  month;
    int  year;
};
```

```
struct Book
{
    char  title[80];
    char  author[80];
    float price;
    char  isbn[20];
};
```

```
struct Library_member
{
    char      name[80];
    char      address[200];
    long      member_number;
    float     fines[10];
    struct Date dob;
    struct Date enrolled;
};
```

```
struct Library_book
{
    struct Book      b;
    struct Date      due;
    struct Library_member *who;
};
```

structure as a function argument

```
#include <stdio.h>
#include <string.h>
struct Books{
    char title[100];
    char author[100];
    char subject[100];
    int book_id;
};

void printBook( struct Books book );    /* function declaration */
int main( ) {
    struct Books Book;    /* Declare Book of type Book */
    strcpy( Book.title, "C omputer Programming");
    strcpy( Book.author, "Professor Engineer");
    strcpy( Book.subject, "C Language Tutorial");
    Book.book_id = 6495407;
    printBook( Book );
return 0;
}

void printBook( struct Books book ) {

    printf( "Book title : %s\n", book.title);
    printf( "Book author : %s\n", book.author);
    printf( "Book subject : %s\n", book.subject);
    printf( "Book book_id : %d\n", book.book_id);
}
```

Array of structure

```
#include<stdio.h>
    struct Employee  {
        int Id;
        int Age;
        char Name[25];
        long Salary;
    };
void main()      {
    int i;
    struct Employee E[ 3 ];
    for(i=0;i<3;i++)  {
        printf("\nEnter detailsof the Employee",i+1);
        printf("\nEnter Employee Id : ");
        scanf("%d",&E[i].Id);
        printf("\nEnter Employee Name : ");
        scanf("%s",&E[i].Name);
        printf("\nEnter Employee Age : ");
        scanf("%d",&E[i].Age);
        printf("\nEnter Employee Salary : ");
        scanf("%ld",&E[i].Salary);
    }
    printf("\nDetails of Employees");
    for(i=0;i<3;i++) {
        printf("\n%d\\%s\\%d\\%ld",E[i].Id,E[i].Name,E[i].Age,E[i].Salary);
    }
}
```

Nested structure

Here, you have outer structure or parent structure and inner structure or child structure.

```
struct stu_address          //inner structure or child structure
{
    int street;
    char *state;
    char *city;
    char *country;
};
struct stu_data             //outer or parent structure
{
    int stu_id;
    int stu_age;
    char *stu_name;
struct stu_address stuAddress; //nested a structure inside another structure.
};
```

Working Example

```
#include <stdio.h>
struct stu_address {
    int street;
    char *state;
    char *city;
    char *country; };
struct stu_data {
    int stu_id;
    int stu_age;
    char *stu_name;
    struct stu_address stuAddress; };
int main(){
    struct stu_data mydata;
    mydata.stu_id = 220019;
    mydata.stu_age = 30;
    mydata.stu_name = "Bob Steve";
    mydata.stuAddress.state = "North";
    mydata.stuAddress.street = 201;
    mydata.stuAddress.city = "North_South";
    mydata.stuAddress.country = "Rwanda";
```

```
printf("Printing student Data: ");  
printf("\nStudent id: %d",mydata.stu_id);  
printf("\nStudent age: %d",mydata.stu_age);  
printf("\nStudent name: %s",mydata.stu_name);  
printf("\nStudent street: %d",mydata.stuAddress.street);  
printf("\nStudent state: %s",mydata.stuAddress.state);  
printf("\nStudent city: %s",mydata.stuAddress.city);  
printf("\nStudent country: %s",mydata.stuAddress.country);  
  
return 0;  
}
```

Union

The major difference between structure and union is:

As discussed, the **structure has a separate memory location for each of its members**

But for union, the members(data types) grouped together **share the same memory location by taking the largest data variable(allocating enough space).**

By using Union in your program, by offering an effective way to use the **same memory location several times** by each data member.

Union in C Programming

Union is a collection of variables of different data types
Union allows to store different data types and size them in the same memory location.

You can define a union with many members, but only one member can contain a value at any given time.

Unions provide an efficient way of using the same memory location for multiple-purpose.

The format of the union statement is as follows

```
union [union tag]  
{  
  member definition;  
  member definition;  
  ...  
  member definition;  
};
```

It is a normal variable definition, such as `int i;` or `float f;` or any other valid variable definition like:

```
union Data {  
  int i;  
  float f;  
  char str[20];  
} data;
```

Working Example

```
#include <stdio.h>
union item {
    int x;
    float y;
    char ch;
};
int main( )
{
    union item it;
    it.x = 22;
    it.y = 10.5;
    it.ch = 'a';
    printf("%d\n", it.x);
    printf("%f\n", it.y);
    printf("%c\n", it.ch);
    return 0; }
```

Working Example

```
#include <stdio.h>
#include <string.h>
union Data {
    int i;
    float f;
    char str[20];
};
int main( ) {
    union Data data;
    data.i = 10;
    printf( "data.i : %d\n", data.i);
    data.f = 220.5;
    printf( "data.f : %f\n", data.f);
    strcpy( data.str, "C Programming");
    printf( "data.str : %s\n", data.str);
    return 0;
}
```

Structure and Union(Allocation Size)

```
#include <stdio.h>
union unionEmployeeInfo
{
char name[32];           //defining a union
float salary;
int workerNo;
} unionEmployeeInfo;
struct structEmployeeInfo
{
char name[32];           //defining sturcture
float salary;
int workerNo;
} structEmployeeInfo;
int main(){
printf("\n Size of union = %d bytes", sizeof(unionEmployeeInfo));
printf("\n Size of structure = %d bytes", sizeof(structEmployeeInfo));
return 0; }
```

```
#include<stdio.h>
struct s_tag
{
    int a;
    long int b;
} x;
union u_tag
{
    int a;
    long int b;
} y;
int main()
{
    printf("Memory allocation for structure = %d", sizeof(x));
    printf("\nMemory allocation for union = %d", sizeof(y));
    return 0;
}
```

Difference Between Structure and Union in C

Structure	Union
Structure size is sum of all member size.	Union size is maximum size of a member variable.
Memory will be allocated for all members.	Memory will be allocated for one member which occupies maximum size.
Data can be manipulated on multiple members properly without losing the content.	If we are manipulating multiple members then actual data is lost.
Multiple members can be initialized.	Only one member required to be initialized.



Exercises

- By using structures,
 1. Make a program which print out names, pages and price of three different books.
 2. Calculation of mean marks of three subjects for 4 students



1. Books

```
#include<stdio.h>
```

```
struct book
```

```
{
```

```
char name[25];
```

```
float price;
```

```
int page;
```

```
};
```

```
main()
```

```
{
```

```
struct book b1, b2,b3;
```

```
printf("Enter the name, price(in dollars) and the number of pages:\n");
```

```
scanf("%s %f %d",&b1.name,&b1.price,&b1.page);
```

```
scanf("%s %f %d",&b2.name,&b2.price,&b2.page);
```

```
scanf("%s %f %d",&b3.name,&b3.price,&b3.page);
```

```
printf("Name=%s\t, Price=%.2f\t, Page= %d\n",b1.name,b1.price,b1.page);
```

```
printf("Name=%s\t, Price=%.2f\t, Page=%d\n",b2.name,b2.price,b2.page);
```

```
printf("Name=%s\t, Price=%.2f\t, Page=%d\n",b3.name,b3.price,b3.page);
```

```
getchar();
```

```
}
```





2. Mean marks calculation

```
#include <stdio.h>
#define N 4
struct student{
    char name[20];
    int eng;
    int math;
    int phys;
    double mean;
};
static struct student data[]={
    {"Jack", 82, 72, 58, 0.0},
    {"Young", 77, 82, 79, 0.0},
    {"Steeve", 52, 62, 39, 0.0},
    {"Mark", 61, 82, 88, 0.0}
};
```



```
main()
{
    int i;
    // Calculation of mean of three subject
    for(i=0; i<N; i++){
        data[i].mean = (data[i].eng + data[i].math +
        data[i].phys)/3.0;
    }
    for(i=0; i<N; i++){
        printf("%s: Eng = %d  Math = %d  Phys = %d: Mean =
        %.1f\n ", data[i].name, data[i].eng, data[i].math, data[i].phys,
        data[i].mean);
    }
    return (0);

}
```





END

